ROS 工业场景小项目 课程报告

基于 MuJoCo 与 ROS 2 的
Go1 四足机器人车间通道通行

选　题：B2 车间通道通行

组　别：第一组

组　长：教晔廷

组　员：臧文韬、牛麒皓、张欣、苏奕婷

指导老师：[指导老师]

代码仓库：github.com/richardzang2006-svg/go1-b2-navigation

日　期：2026 年 10 月

摘　要

本项目完成课程参考选题 B2“车间通道通行”：让 Unitree Go1 四足机器人在仿真车间中从起点自主到达目标点，并在遇到新增障碍时绕行。系统以 MuJoCo 作为物理仿真平台，以 ROS 2 Jazzy 的 Navigation2（Nav2）作为导航栈，底层步态复用已训练好的强化学习行走策略。我们搭建了 27 m × 18 m 的工厂车间场景，其中包含带 4 个 180° 转弯的货架迷宫和 6 处不在导航地图上的临时障碍物；编写了指令看门狗与急停、任务下发与进度反馈、启动守护三个 ROS 2 节点；并用脚本从同一份碰撞几何同时生成仿真场景和导航地图，同时自动验证机器人模型在改造过程中保持不变。

在无界面模式下，从起点到迷宫终点（规划路径约 49.5 m）的两次完整运行均成功到达，最快导航用时约 44 s，最高速度约 1.29 m/s，全程未摔倒、未触发恢复动作。报告最后分析了调试中遇到的问题，以及遇障状态反馈、台阶通行等后续改进方向。

关键词：ROS 2；Nav2；MuJoCo；四足机器人；强化学习步态；局部避障

1 任务与需求

1.1 工业场景

车间里设备与物料之间的通道是移动机器人最常见的作业空间：通道窄、转弯多，而且经常出现临时堆放的托盘、推车和警示锥桶，这些物体不在预先建好的地图上。四足机器人能适应比轮式底盘更复杂的地面，但它的步态控制本身就比较困难。本项目在仿真中复现这样一个车间，让四足机器人完成“从起点安全到达目标点”的通行任务。

1.2 任务定义

表 1 任务定义

任务要素	本项目
输入	目标点（x, y, 朝向），由 goal.py 自动发送，或在 RViz 中用“Nav2 Goal”点选
机器人动作	Go1 从起点出发，沿主通道进入货架迷宫，绕开地图外的障碍物，到达终点
结果判断	Nav2 返回 SUCCEEDED / FAILED；同时记录轨迹、用时、机身高度和离障碍物的最近距离

1.3 B2 要求对照

表 2 B2 要求完成情况

B2 要求	本项目实现	状态
从起点到达目标点	起点 (0, 0) 到迷宫终点 (6.5, 8.5)，规划路径约 49.5 m，含 4 个 180° 转弯	完成
遇到新增障碍时停车或绕行	6 处地图外障碍，由局部规划绕行；碰撞监控预测到碰撞时减速	完成（绕行）
距离感知	90 线 2D 激光，10 Hz，接入代价地图、定位和碰撞监控	完成
速度或导航指令	Nav2 输出速度，经平滑、碰撞监控、看门狗后发送给机器人	完成
到达 / 遇障状态反馈	/workshop/mission 发布剩余距离与 SUCCEEDED / FAILED；遇障状态尚未单独发布	部分完成
拓展：重新规划	全局代价地图 5 Hz 刷新，新障碍会进入全局路径重新计算	完成

2 技术选型

2.1 仿真平台：MuJoCo

项目早期使用 Gazebo + CHAMP 驱动 Unitree Go2：步态由 CHAMP 规划器生成，参数需要手工调整，导航限速只有 0.5 m/s，世界文件与导航地图也分开维护。后来改为 MuJoCo + 强化学习策略，主要原因有三点：

- **训练与部署必须同引擎**。本项目复用的 Go1 行走策略是在 MuJoCo 中训练的（mujoco_playground + MJX + brax PPO，约 2 亿步）。据策略作者记录，曾尝试 Isaac Gym 训练、MuJoCo 部署，由于两个引擎的接触模型不同，策略在 0.3 m/s 以上就会失稳；改为同引擎训练后问题消失。
- **可编程建模**。MuJoCo 的 MjSpec 接口可以用 Python 增删物体、读写参数，整座工厂由脚本生成，修改布局只需改代码。
- **射线检测**。mj_ray 可以直接用来实现 2D 激光雷达，也可以把场景光栅化成导航地图。

2.2 机器人：Unitree Go1

- **有现成的行走策略**。附带训练好的策略权重和观测归一化参数，推理代码与训练框架逐位比对，最大误差 1.2×10^{-7}，无需从零训练。
- **模型质量高**。机器人模型来自 DeepMind MuJoCo Menagerie（BSD-3 许可），质量、惯量、关节限位完整；整机约 12.7 kg、12 个关节，适合 3 m 宽的通道。
- **接口标准**。运动层对外只使用 /cmd_vel、/odom、/scan 等标准话题，将来迁移实机时只需把运动节点换成基于 unitree_ros2 的版本。

3 系统设计

3.1 总体架构

系统分为四层，形成闭环（图 1）。导航层 Nav2 负责定位、路径规划与避障；安全层 workshop_runtime 负责指令超时停车和急停；运动层 go1_locomotion 用强化学习策略把速度指令转换为 12 个关节的目标角；物理层 MuJoCo 完成动力学仿真和激光射线检测。里程计、激光和坐标变换再反馈回导航层。仿真场景和导航地图在运行前由脚本离线生成，保证二者一致。



图 1 系统四层架构（橙色为本组编写的模块）

3.2 ROS 2 通信关系

图 2 给出核心节点与话题（省略了 /rosout 等辅助话题）。速度指令依次经过速度平滑、碰撞监控和看门狗三道环节才到达机器人：controller_server 发布 /cmd_vel_nav，velocity_smoother 输出 /cmd_vel_smoothed，collision_monitor 输出 /cmd_vel，workshop_runtime 输出 /cmd_vel_safe，最终由 go1_locomotion 执行。反馈方向上，go1_locomotion 发布 /odom 和 /scan，AMCL 结合 map_server 发布的 /map 完成定位，并发布 map→odom 坐标变换。

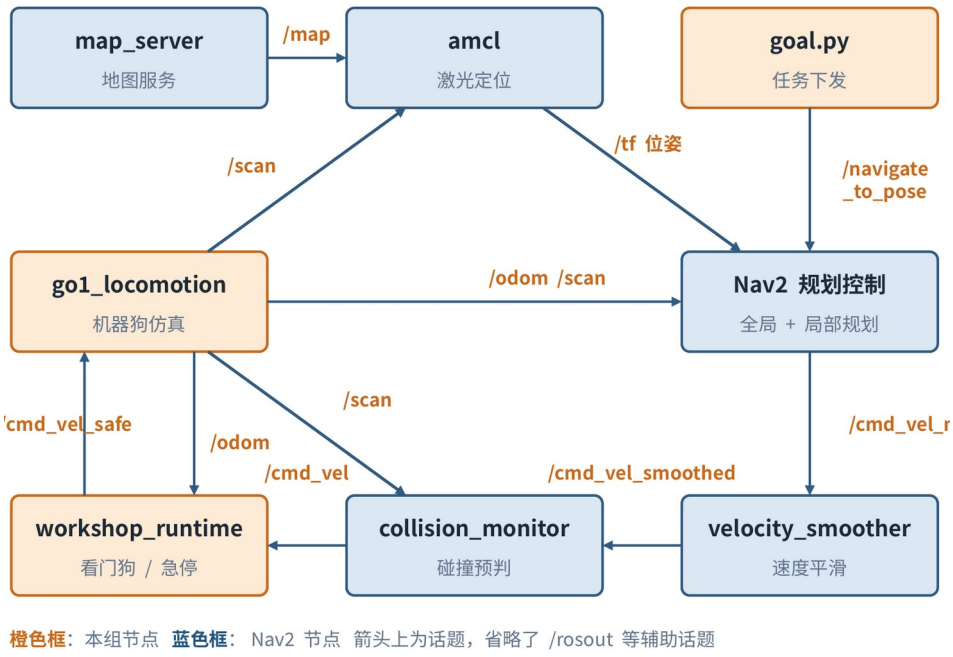


图 2 核心节点与话题通信关系

表 3 主要 ROS 2 节点

节点	来源	作用
go1_locomotion	复用并集成	MuJoCo 仿真 + RL 策略推理；订阅 /cmd_vel_safe，发布 /odom、/scan、/joint_states
workshop_runtime	本组编写	0.5 s 指令看门狗、急停服务；发布 odom→base_link 坐标变换和运行状态
goal.py（basic_navigator）	本组编写	等待 Nav2 就绪后发送目标；发布 /workshop/mission 任务进度
lifecycle_guard	本组编写	检查并补全卡住的 Nav2 生命周期启动
map_server / amcl	Nav2	加载工厂地图；粒子滤波定位
planner_server / controller_server	Nav2	NavFn 全局规划；DWB 局部规划输出速度
velocity_smoother	Nav2	限制速度与加速度，使指令平滑
collision_monitor	Nav2	根据激光预测 1.2 s 内的碰撞并减速
bt_navigator / behavior_server	Nav2	行为树调度导航任务与恢复动作

4 关键实现

4.1 工厂场景构建与验证

原始模型 go1_corridor.mjb 是编译后的 MuJoCo 二进制文件，不能直接追加物体。我们编写 build_go1_factory.py，从编译模型中逐项读出刚体、惯量、关节、执行器、网格、关键帧和地面接触参数，重建为可编辑的 MjSpec，再在外部搭建工厂：墙体、货架迷宫、传送带、4 台加工设备、叉车、卷帘门和地面标线，全场共 1017 个几何体（图 3）。

由于行走策略对物理参数非常敏感，参数改错不会报错，只会让机器人走歪，因此脚本在保存前做两步验证：一是静态检查，机器人相关的全部参数与原模型一致；二是动态检查，让同一策略在新旧两个模型中各运行 8 s（直行、转弯各一次），两边轨迹必须完全相同。实测轨迹偏差为 0 m，即逐位一致。

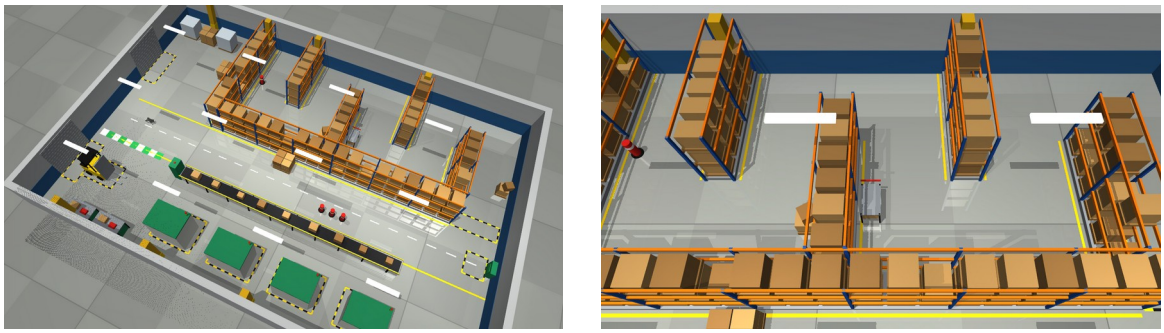


图 3 仿真车间全景（左）与蛇形货架迷宫（右）

几何体分为三组，用来区分“地图里有什么”和“传感器能看到什么”。地图生成脚本 generate_go1_factory_map.py 从 2.95 m 高处向下逐格做射线检测，只统计第 0 组，得到 580 × 400 格、5 cm 分辨率的 Nav2 地图。

表 4 几何体分组

分组	内容	参与碰撞	激光可见	进入地图
0	墙、货架、设备等固定障碍	是	是	是
1	地面标线、灯具等装饰	否	—	否
2	托盘、锥桶、货箱、推车等临时障碍	是	是	否

早期的 S 形迷宫转弯开口只有 2.1 m，扣除机身半径 0.35 m 和膨胀区后几乎没有可通行空间，机器人会卡在掉头处。新迷宫的通道与转弯开口统一为 3 m，临时障碍物旁至少留出 2.1 m 的通行宽度。

4.2 运动控制

go1_locomotion 节点以 50 Hz 运行强化学习策略，物理仿真步长 4 ms（250 Hz）。策略输入 48 维观测：机身线速度与角速度、重力方向、12 个关节的角度偏差与角速度、上一步动作和速度指令 (vx, vy, ωz)；观测先做均值方差归一化，经过 48→512→256→128→24 的全连接网络（swish 激活、tanh 输出）后取前 12 维作为动作。关节目标角 = 动作 × 0.5 + 默认站姿，由 MuJoCo 的 PD 位置执行器（kp 35，kd 0.5）跟踪。

我们在空旷平地上测试了策略的速度跟踪能力（表 5，取第 5～10 s 的稳态平均）。低于 0.3 m/s 时跟踪很差（据原文档，0.09 m/s 指令的跟踪率仅约 1%），因此

Nav2 的最小速度设为 0.28 m/s、最小角速度设为 0.6 rad/s。1.5 m/s 已超出原文档给出的 1.0 m/s 范围，但在直行、转弯以及急停、急转、再加速的连续序列中均未摔倒，因此导航速度上限设为 1.5 m/s。该结论只在平整地面上验证过。

表 5 平地速度跟踪实测

指令（前进，转向）	实际速度
0.8 m/s	0.79 m/s
1.0 m/s	1.01 m/s
1.2 m/s	1.22 m/s
1.5 m/s	1.51 m/s
1.5 m/s，1.0 rad/s	1.45 m/s

4.3 导航与避障

全局规划器 NavFn 只依据静态地图，因此最初规划出的路径会直接穿过地图外的障碍物。机器人接近障碍时，激光把它标记进局部代价地图，DWB 局部规划器每秒 8 次在约 220 组候选速度中按“贴合全局路径、远离障碍、朝向目标”打分，选出绕行轨迹；同时全局代价地图以 5 Hz 刷新，新障碍进入全局规划，路径随之重算。主要参数见表 6。

表 6 Nav2 主要参数

参数	取值
速度范围	前进 0.28～1.5 m/s，转向 0.6～1.0 rad/s
机身半径 / 膨胀半径	0.35 m / 0.75 m
局部代价地图	7 m × 7 m，5 Hz
全局代价地图	5 Hz 刷新
局部规划器	DWB，8 Hz，线速度 20 个采样、角速度 11 个采样
定位	AMCL 似然场模型，500～2000 个粒子

4.4 安全与可靠性

- **碰撞监控：**collision_monitor 根据激光预测未来 1.2 s 内是否会碰撞，预测到碰撞时提前减速。

- **指令看门狗：**workshop_runtime 在 0.5 s 内没有收到速度指令或里程计时，自动输出零速度，上层程序异常时机器人也会停下。
- **急停服务：**通过 /workshop/emergency_stop（std_srvs/SetBool）一条命令立即停车，解除后继续任务。
- **启动守护：**测试中发现 Fast DDS 偶尔会丢失一条生命周期服务应答，导致 AMCL 未被激活、没有 map→odom 变换，机器人原地不动。我们没有修改 Nav2 源码，而是新增 lifecycle_guard 节点：启动 20 s 后逐个检查 Nav2 节点，把卡住的节点推到 active 状态。

4.5 任务下发与状态反馈

goal.py 等待 Nav2 就绪后发送 /navigate_to_pose 目标，运行中每秒在 /workshop/mission 发布剩余距离，结束时发布 SUCCEEDED、FAILED 或 CANCELED；workshop_runtime 在 /workshop/status 发布 MOVING、STOPPED、EMERGENCY_STOP 状态，并在 RViz 中以文字标记显示。整套系统通过 ./scripts/start_demo.sh 一条命令启动，Nav2 就绪后自动出发。

5 实验与结果

测试在无界面模式下进行（2026 年 10 月 19 日），目标为迷宫终点 (6.5, 8.5)，规划路径约 49.5 m。第 1 次运行后，我们把主通道托盘后移 2 m、全局代价地图刷新从 1 Hz 提到 5 Hz、膨胀半径从 0.55 m 加到 0.75 m，第 2 次为当前配置。

表 7 完整运行测试结果

指标	第 1 次	第 2 次（当前配置）
导航结果	SUCCEEDED	SUCCEEDED
导航用时	约 50 s	约 44 s
恢复动作（原地旋转、后退等）	0 次	0 次
离托盘最近（机身中心）	0.27 m（过近）	0.37 m
离其余 5 处障碍最近	1.02～1.98 m	0.95～1.87 m
机身最低高度	0.282 m	0.282 m

两次运行均到达终点，全程无摔倒，机身高度保持在 0.28～0.32 m；第 1 次运行的最高速度（1 s 平滑）约 1.29 m/s。第 2 次运行中机身中心离托盘 0.37 m，腿部离托盘约 0.17 m，不会碰撞。由于完整测试只有两次，结论的统计意义有限。

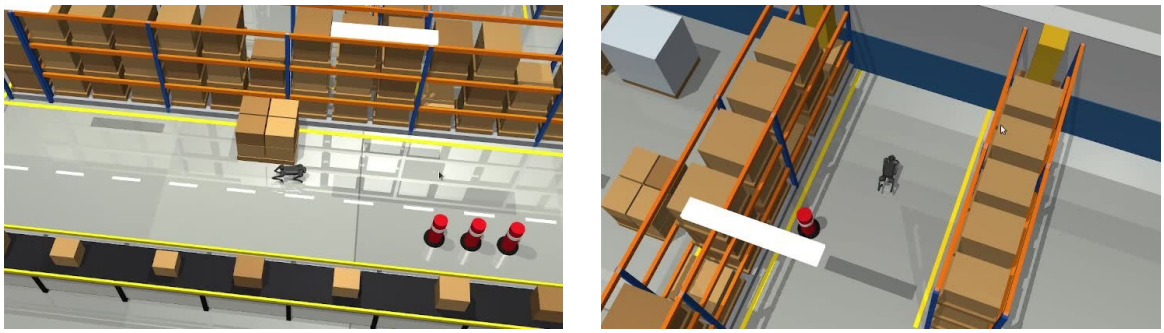


图 4 演示视频截图：主通道绕过托盘和锥桶（左），在货架迷宫中掉头前进（右）

6 问题与解决

表 8 调试中遇到的问题

现象	原因	解决办法
迷宫掉头处卡住	早期迷宫转弯开口仅 2.1 m，扣除机身和膨胀区后几乎无路	通道与转弯开口统一放宽到 3 m
启动后原地不动	Fast DDS 偶发丢失生命周期应答，AMCL 未激活	新增 lifecycle_guard 自动补全启动
机器人在躲“看不见的东西”	临时障碍放在 MuJoCo 查看器默认隐藏的几何组 4	改到默认可见、但不进地图的第 2 组
擦着托盘经过	全局代价地图 1 Hz 刷新，路径更新太晚	改为 5 Hz、膨胀 0.75 m，距离从 0.27 m 增至 0.37 m
低速指令时不走	策略在 0.3 m/s 以下有跟踪死区	Nav2 最小速度 0.28 m/s、最小角速度 0.6 rad/s

7 不足与展望

- **遇障状态反馈：**目前只发布到达状态，尚未单独发布“遇障减速 / 停车”状态，也没有演示通道被完全堵死时的停车处理。下一步计划在 workshop_runtime 中根据碰撞监控状态和激光最近距离发布遇障状态。
- **台阶通行：**我们测试过现有策略：1.5 cm 以下的台阶可以通过，2 cm 起全部失败，原因是策略只用本体感受、只在平地训练。要实现台阶通行，需要补全腿部

与机身碰撞体、加入从低到高的地形难度课程，并训练带高度扫描或教师-学生蒸馏的地形感知策略。

- **动态障碍：**加入移动的叉车和行人，测试动态避障与重新规划。
- **实机迁移：**用 unitree_ros2 替换运动节点；观测中的机身线速度在实机上无法直接测量，需要状态估计。
- **测试规模：**完整运行只有两次，后续应增加不同目标点和障碍布局的重复测试。

8 成员分工与资料来源

8.1 成员分工

第一组成员：教晔廷（组长）、臧文韬、牛麒皓、张欣、苏奕婷。按课程给出的五个角色分工如下。

表 9 成员分工

角色	成员	负责内容
A 仿真与平台	[姓名]	工厂场景生成与验证、启动配置
B 动作与执行	[姓名]	运动节点集成、Nav2 参数与速度调优
C 感知与交互	[姓名]	激光检测、代价地图、障碍物分组与地图生成
D 任务与记录	[姓名]	任务流程、看门狗与急停、运行记录
E 集成与测试	[姓名]	启动守护、联调测试、演示视频与文档

8.2 自行完成与复用部分

表 10 自行完成与复用部分

类别	内容
本组完成	工厂场景与地图生成脚本（含机器人一致性验证）；workshop_runtime 看门狗与急停；goal.py 任务下发与进度反馈；lifecycle_guard 启动守护；Nav2 参数调优；一键启动脚本与测试记录
复用开源	Go1 机器人模型（MuJoCo Menagerie）；Go1 行走策略与运动节点（随项目提供的 go1_mujoco_locomotion 包，训练来源未独立核实）；ROS 2 Jazzy；Navigation2；MuJoCo
早期参考（未使用）	CHAMP 四足控制框架；Unitree Go2 ROS 2 仿真包

9 结论

本项目在 MuJoCo 仿真中实现了 B2“车间通道通行”：Go1 四足机器人可以在 27 m × 18 m 的车间里，从起点出发穿过带 4 个掉头的货架迷宫，绕开地图上没有的障碍物，自主到达终点。得到的主要经验是：学习得到的步态对物理引擎很敏感，训练和部署必须使用同一引擎；仿真场景和导航地图应由同一份几何生成，并在每次修改后自动验证机器人没有被改动；速度指令应分层把关，由碰撞监控、看门狗和急停共同保障安全。不足之处是只能在平地上行走、只在仿真中验证、遇障状态反馈尚未完成，这些是下一步的工作方向。

附录　运行说明

代码仓库：https://github.com/richardzang2006-svg/go1-b2-navigation（README 含完整说明）。运行环境为 Ubuntu 24.04、ROS 2 Jazzy（含 Nav2）、Python 3.12、MuJoCo 3.x。

```
cd ~/ros2_ws && source scripts/env.sh && colcon build --symlink-install
./scripts/start_demo.sh                        # 自动发送目标并出发
./scripts/start_demo.sh auto_goal:=false       # 在 RViz 中手动选择目标
ros2 service call /workshop/emergency_stop std_srvs/srv/SetBool '{data:
true}'   # 急停
```

参考资料

[1] MuJoCo 物理引擎文档：https://mujoco.readthedocs.io

[2] MuJoCo Menagerie（Unitree Go1 模型）：https://github.com/google-deepmind/mujoco_menagerie

[3] MuJoCo Playground：https://github.com/google-deepmind/mujoco_playground

[4] ROS 2 Jazzy 文档：https://docs.ros.org/en/jazzy

[5] Navigation2 文档：https://docs.nav2.org

[6] CHAMP 四足控制框架：https://github.com/chvmp/champ